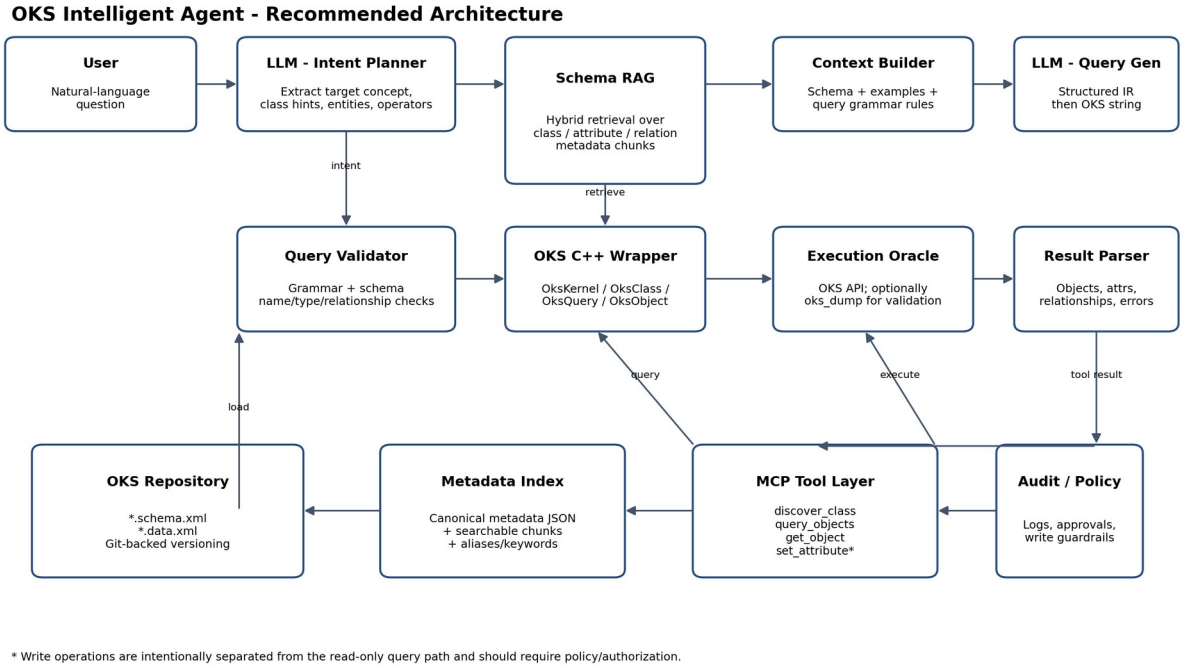
OKS INTELLIGENT QUERY AGENT

Technical Architecture & Codex Implementation Specification

ATLAS TDAQ / DUNE-DAQ Object Kernel Support (OKS)

Prepared from the three supplied OKS technical references
Implementation target: Python MCP + Schema RAG + LLM + deterministic C++ OKS wrapper



* Write operations are intentionally separated from the read-only query path and should require policy/authorization.

Source-of-truth rule
This document is an implementation specification, not a claim that every deployment exposes the same schema inventory. The supplied references explicitly state that concrete schema inventory was incomplete in places; the live OKS installation and loaded schema remain authoritative.

Version: 1.0 | 13 August 2026

Table of Contents

1. Executive Objective

Build an AI agent that accepts natural-language questions about an OKS configuration repository, identifies the relevant OKS classes/attributes/relationships, generates a valid OKS query, validates it against the authoritative schema/parser, executes it through a deterministic C++ OKS layer, and turns the returned objects into a grounded answer.

- The LLM is responsible for semantic interpretation and query planning; it should not generate arbitrary C++ source code for each request.
- The live OKS schema/data repository is the source of truth for classes, attributes, relationships, object IDs, types, ranges, inheritance, and current configuration state.
- RAG is used to retrieve relevant schema metadata and query-language examples/rules into the LLM context. RAG is not the execution mechanism.
- The final OKS query is validated by the actual parser / OKS API before execution.
- For production, keep read-only querying separate from write/configuration operations. Writes must be policy-controlled and auditable.

Why this architecture
The supplied learning guide explicitly recommends structured query generation: natural language -> identify class/attributes/relationships/IDs/time context -> build an intermediate query tree -> serialize to OKS grammar -> validate -> execute -> interpret results. It also notes that no official NL-to-OKS corpus was found, so a project dataset must be machine-checked against the real parser.

2. Source Grounding and Scope

This specification is grounded in the three user-supplied PDFs. Where the references distinguish verified source behavior from illustrative examples, this document preserves that distinction.

Source	Role in this specification
OKS_DAL_Technical_Reference.pdf	System overview, OKS/DAL layering, XML storage, schema examples, versioning, CLI, gold-pair corpus, provenance and limitations.
OKS_Grammar_Query_CppAPI_Reference.pdf	Query grammar/parser behavior, public C++ API surface, query execution, object/attribute/relationship APIs, CLI/API details and release notes.
OKS_Query_Language_Learning_Guide_Detailed(1).pdf	Teaching-oriented query grammar, semantic decomposition, intermediate-tree approach, validation workflow, MCP implications and limitations.

2.1 What is authoritative at runtime?

- The currently loaded OKS schema and data files.
- The installed OKS parser/API version.
- Repository/version selection when historical configuration is requested.
- The actual class/attribute/relationship names returned by the live repository, not a hard-coded inventory from training data.

3. OKS Mental Model

OKS is an object manager centered on uniquely identified objects. Four concepts matter for the agent: classes, attributes, relationships, and objects.

Concept	Meaning	Agent use
Class	Schema-level definition of a type; may be abstract and may inherit from superclasses.	Candidate target class for a user request.
Attribute	Typed field on a class; may have default, range, nullability, multi-value and ordering metadata.	Leaf predicate and value/type constraint.
Relationship	Typed reference from an object to one or more objects; has cardinality and ownership flags.	Traversal condition and graph navigation.
Object	Concrete instance with unique object ID, attribute values and relationship references.	Actual result entity or target of a read/write operation.

Example mental model

```
Class: RCApplication
  superclass: Application
  attributes: Name, Timeout
  relationship: ApplicationsControlled -> Application

Object: rc_readout_1
  Name = "Readout"
  Timeout = 20
  ApplicationsControlled -> { ... }
```

Schema vs data
The supplied references explicitly distinguish schema files (*.schema.xml) from data files (*.data.xml): schema defines classes; data files contain concrete objects. Includes allow site-specific schemas to build on shared schemas.

4. OKS Schema and Metadata Model

The schema is the central knowledge source for NL-to-OKS generation. The agent should transform the live schema into a machine-readable metadata model and index that model for retrieval.

4.1 Schema-level metadata to extract

Field	Required?	Why it matters
class_name	Yes	Maps user intent to a legal OKS class.
description	Recommended	Semantic retrieval and disambiguation.
is_abstract	Yes	Prevents selecting a class that cannot directly have concrete instances.
superclasses	Recommended	Inheritance-aware retrieval and scope selection.
attributes[]	Yes	Available predicates and writable fields.
attribute.type	Yes	Validation and value construction.
attribute.range	When present	Enum/range validation.
attribute.init_value	When present	Defaults/sanity checks.
attribute.is_not_null	Yes	Validation constraint.
attribute.is_multi_value	Yes	Scalar vs collection semantics.
attribute.ordered	When present	Collection semantics.
relationships[]	Yes	Graph traversal/query construction.
relationship.class_type	Yes	Target class resolution.
relationship.low_cc/high_cc	Yes	Cardinality-aware operations and some/all semantics.
relationship.is_composite/is_exclusive/ is_dependent	When present	Write and lifecycle policy.

source_file/includes	Recommended	Traceability and repository context.
repository_version	Recommended	Reproducibility and historical context.

4.2 Attribute types exposed by the OKS XML schema DTD

The reference reproduces the schema DTD attribute type enumeration: bool, s8, u8, s16, u16, s32, u32, s64, u64, float, double, date, time, string, uid, enum and class. The DTD also describes range, format, multi-value, init-value, nullability and ordering.

```
Canonical metadata JSON - recommended
{
 "class": "RCApplication",
 "description": "An executable which allows users to control datataking",
 "abstract": false,
 "superclasses": ["Application"],
 "attributes": [
  {
   "name": "Timeout",
   "description": "Seconds to wait before giving up on a transition",
   "type": "u16",
   "range": "1..3600",
   "init_value": "20",
   "is_not_null": true,
   "is_multi_value": false
  }
 ],
 "relationships": [
  {
   "name": "ApplicationsControlled",
   "target_class": "Application",
   "low_cc": "one",
   "high_cc": "many"
  }
 ],
 "source_file": "tutorial.schema.xml"
}
```

5. OKS Query Language Specification

The query language is a parenthesized, Lisp-like S-expression grammar. It expresses scope, Boolean composition, attribute comparisons, relationship conditions, object-ID tests, and path queries.

5.1 Core grammar

```
query ::= "(" scope expr ")"
scope ::= "this" | "all"
expr ::= attr_cmp | uid_cmp | rel_expr | and_expr | or_expr | not_expr
and_expr ::= "and" expr+
or_expr ::= "or" expr+
not_expr ::= "not" expr
attr_cmp ::= "(" "\"attr-name\"" "\"value\"" op ")"
uid_cmp ::= "(" "object-id" "\"an-object-id\"" "=" ")"
rel_expr ::= "(" "\"rel-name\"" ("some"|"all") expr ")"
```

```
path_query ::= (path-to "\"dest-id@class\"" (direct|nested "\"rel\"" ...))
op ::= "=" | "!=" | "~= " | "<" | "<=" | ">" | ">="
```

Grammar provenance
The references explicitly label this BNF as reconstructed because there is no formal grammar file in the repositories; src/query.cpp, include/oks/query.hpp comments, QueryWindow documentation and release notes are the authoritative grounding.

```
Canonical comparators
=   equality
!=  inequality
~=  regular-expression match
<   less than
<=  less than or equal
>   greater than
>=  greater than or equal
```

5.2 Scope

```
(this ("Status" "Running" =))
(all ("Status" "Running" =))
```

this limits the search to the named class; all includes the class and its subclasses. The word all also appears inside relationship expressions, where it means every referenced object rather than class scope.

5.3 Attribute comparisons

```
(all ("Max Speed" "200" >))
```

Values are written as quoted strings in query text; the parser later interprets them using the target attribute metadata at evaluation time. The supplied references emphasize this weak typing.

5.4 Boolean expressions

```
(and e1 e2 ...)
(or e1 e2 ...)
(not e)

Example:
(all
 (and
  ("Status" "Running" =)
  ("Priority" "High" =)))
```

and/or require two or more operands; not requires exactly one operand.

5.5 Relationship expressions

```
("RunsOn" some (object-id "lxplus001.cern.ch" =))
("RunsOn" all  (object-id "hostB" =))
```

some means at least one related object satisfies the nested condition. all means every related object satisfies it.

5.6 Object-ID comparisons

```
(object-id "lxplus001.cern.ch" =)
```

The object-id pseudo-attribute is restricted to equality in the documented grammar and is commonly nested in relationship expressions.

5.7 Path queries

```
(path-to "my-id@my-class"
 (direct "A" "B"
  (nested "N"
   (direct "X" "Y" "Z"))))
```

Path queries describe topology: they search for relationship chains toward a destination object. They are not simple attribute filters.

6. Natural Language -> OKS Query Pipeline

The agent should never rely on word-for-word translation. The request must first be converted into a logical representation that preserves scope, Boolean structure, relationship direction, quantification, and the object at which each condition is evaluated.

```
User question
  |
  v
Semantic intent extraction
  |
  +-- target class hint
  +-- attribute / relationship concept
  +-- operator
  +-- values / IDs / time context
  +-- requested output
  |
  v
Schema retrieval
  |
  v
Intermediate query tree (IR)
  |
  v
OKS S-expression serializer
  |
  v
Authoritative parser validation
  |
  v
OksClass::execute_query / find_path
  |
  v
Result normalization -> LLM answer
```

6.1 Example

Question: “Which applications run on lxplus001.cern.ch?”

Semantic component	Resolved concept
Target	Application
Action	find matching objects
Relationship	RunsOn

Quantifier	some
Related object	lxplus001.cern.ch
Identity test	object-id =

```
(all ("RunsOn" some (object-id "lxplus001.cern.ch" =)))
```

7. Schema RAG Architecture

RAG is appropriate here for schema grounding because the repository can contain many classes, inherited attributes, relationships, descriptions, and version-dependent configurations. The important rule is that RAG supplies context; the OKS engine remains the execution authority.

7.1 Two-stage retrieval

1. Stage A - intent keyword generation. An LLM planner converts the user question into retrieval terms such as domain, class candidates, attribute concepts, relationship concepts, object identifiers, units and operational keywords.
2. Stage B - schema retrieval. A hybrid index retrieves schema chunks matching those terms, then a context builder returns only the relevant classes, inherited members, relationships, and validation constraints.

```
Example:
User: "Find applications running on hostA with timeout > 25"

Planner keywords:
  class_hints = [Application, RCApplication]
  attribute_hints = [Timeout]
  relationship_hints = [RunsOn]
  object_id_hints = [hostA]
  comparator_hints = [>]

Retriever returns:
  Application class metadata
  RCApplication superclass/attributes
  RunsOn relationship metadata
  Timeout type/range
  Relevant English->OKS examples
  Grammar constraints for some + object-id + >
```

7.2 What to index

Chunk type	Example contents	Retrieval key
Class chunk	class name, description, superclasses	class semantic + name
Attribute chunk	class, attribute, type, range, description	attribute concept + class
Relationship chunk	source class, relation, target class, cardinality	relation verbs/nouns + source/target
Object summary chunk	class, object ID, selected searchable attrs	entity/object terms
Query example chunk	English question + valid OKS query	semantic pattern + grammar construct
Version chunk	repository tag/hash/date semantics	run, version, revision, date

7.3 Hybrid retrieval is preferred

- Vector similarity for semantic terms such as “host the application runs on” -> RunsOn.
- Lexical/keyword retrieval for exact identifiers such as `RCApplication`, `ApplicationsControlled`, or object IDs.
- Metadata filters for class name, repository version, schema file, and type.

- Optional reranking using the planner's extracted class/relationship constraints.

RAG indexing rule
Do not embed entire XML files as undifferentiated documents. Parse them into structured metadata first. The index should preserve class/attribute/relationship boundaries so the context builder can assemble a coherent schema slice.

8. Intermediate Query Representation (IR)

The IR is the safety boundary between language understanding and OKS syntax. It lets the agent validate structure before serializing a string.

```
{
 "scope": "all",
 "expression": {
  "type": "and",
  "operands": [
   {
    "type": "attribute_compare",
    "attribute": "Status",
    "operator": "=",
    "value": "Running"
   },
   {
    "type": "relationship",
    "name": "RunsOn",
    "quantifier": "some",
    "expression": {
     "type": "object_id",
     "operator": "=",
     "object_id": "hostA"
    }
   }
  ]
 }
}
```

8.1 IR validation rules

- Only scope values `this` or `all` are accepted.
- Attribute names must exist on the selected class (including inherited attributes when the runtime API exposes them that way).
- Relationship names must exist on the selected class and their target class must resolve.
- `some` / `all` relationship quantifiers require a nested expression.
- `not` has exactly one operand.
- `and` and `or` have at least two operands.
- Object-ID tests use equality only.
- Path queries require a destination object in `id@class` form and valid relationship path syntax.
- Values must be compatible with the schema attribute type/range before query execution.

9. MCP Tool Contracts

The LLM should call narrowly defined tools rather than write C++ source code or shell commands.

9.1 Recommended tools

Tool	Purpose	Read/Write
list_classes	Enumerate available classes from live schema.	Read
describe_class	Return class metadata, inheritance, attributes, relationships.	Read
find_objects	Find object IDs by class and optional filters.	Read
get_object	Return an object's attributes and selected relationships.	Read
query_objects	Execute validated OKS query against a class.	Read
find_path	Execute an OKS path query from a starting object.	Read
get_repository_version	Resolve current/historical repository version.	Read
set_attribute	Modify one object attribute after policy checks.	Write-controlled
set_relationship	Modify an object relationship after policy checks.	Write-controlled

9.2 Example `query_objects` contract

```
Input
{
 "class": "Application",
 "query_ir": { ... }
}

Output
{
 "ok": true,
 "query_oks": "(all (\"RunsOn\" some (object-id \"hostA\" )))",
 "class": "Application",
 "objects": [
  {"id": "rc_trigger_1", "attributes": {"Name": "..."}}
 ],
 "count": 1,
 "repository_version": "hash:..."
}
```

9.3 Example `set_attribute` contract

```
Input
{
 "class": "RCApplication",
 "object_id": "rc_trigger_1",
 "attribute": "Timeout",
 "value": 25
}

Required server-side checks:
```

1. class exists
2. object exists and belongs to class hierarchy
3. attribute exists and is writable
4. type/range/nullability constraints pass
5. policy/authorization permits the mutation
6. audit log records before/after + repository version
7. persistence succeeds

10. C++ OKS Wrapper Design

The C++ wrapper is the deterministic execution layer. The LLM should provide structured parameters; the wrapper performs repository loading, class/object/attribute lookup, query construction, execution and serialization of results.

10.1 Core API surface identified in the supplied reference

API area	Relevant methods / concepts
OksKernel	load_schema(), load_data(), load_file(), find_class(), classes(), objects(), repository root/version APIs.
OksClass	all_attributes(), direct_attributes(), find_attribute(), all_relationships(), find_object()/get_object(), execute_query().
OksObject	GetId(), GetClass(), GetAttributeValue(), GetRelationshipValue(), SetAttributeValue(), relationship add/remove, find_path().
OksQuery	string parser, OksComparator, OksAndExpression, OksOrExpression, OksNotExpression, OksRelationshipExpression.
OksFile	schema/data file representation, includes, locking and repository/file operations.

10.2 Metadata extraction flow

```
OksKernel kernel;
load repository/schema/data
    |
    +--> kernel.classes()
    |    |
    |    +--> class.get_name()
    |    +--> class.get_description()
    |    +--> class.all_super_classes()
    |    +--> class.all_attributes()
    |    +--> class.all_relationships()
    |
    +--> each object: GetId(), attributes, relationships
    |
    v
canonical metadata JSON -> RAG index
```

10.3 Programmatic query path

```
OksClass* c = kernel.find_class(class_name);
OksAttribute* a = c->find_attribute(attribute_name);
OksComparator cmp(a, new OksData(value_as_oks_string_or_typed_data), comparator);
OksQuery q(false, &cmp_or_expression);
auto* result = c->execute_query(&q);
```

Do not hard-code an unverified API signature
The exact ownership/constructor choice must match the installed OKS headers. The reference demonstrates OksComparator + OksData + OksQuery and execute_query(), and separately documents the string-query constructor. Codex should inspect the installed headers before finalizing memory ownership or helper wrappers.

11. Query Validation and Safety

Validation must happen before execution. Syntax-valid is not the same as schema-valid, and schema-valid is not the same as semantically appropriate.

Layer	Check	Failure example
1. JSON / IR schema	Required fields, enums, types.	operator missing
2. Schema validation	Class, attribute, relationship and target class exist.	unknown relationship RunsOnX
3. Semantic validation	Requested value is compatible with attribute meaning/type/range.	Timeout=50000 outside 1..3600
4. Grammar validation	Serialized S-expression parses.	unbalanced parentheses
5. Repository integrity	No dangling references / broken includes.	query execution blocked by invalid DB
6. Execution	OKS query actually evaluates.	oks::QueryFailed
7. Result validation	Cardinality, result count, expected object class.	0 results for a supposedly unique target

Recommended validation pipeline

```
NL question
  -> Planner
  -> Schema RAG
  -> Query IR
  -> IR validator
  -> OKS string serializer
  -> OksQuery parser / q.good()
  -> schema/name checks
  -> execute_query()
  -> result checks
  -> answer
```

12. Read Path End-to-End

3. Receive user question.
4. LLM planner emits retrieval keywords and a provisional semantic intent; no OKS code is executed yet.
5. Retrieve the relevant schema chunks and a small set of verified query examples.
6. Resolve class, attribute, relationship and object ID against the live schema/data.
7. Build the query IR.
8. Serialize IR to OKS S-expression.
9. Validate via the actual parser / C++ wrapper.
10. Execute via OksClass::execute_query() or OksObject::find_path().
11. Normalize OksObject results into stable JSON.
12. Pass only the returned objects plus provenance/version back to the answering LLM.
13. LLM generates a grounded answer; it must not invent values absent from the result set.

13. Write / Configuration Path (Controlled)

The supplied references confirm that OksObject exposes SetAttributeValue and relationship mutation methods, and that DAL provides higher-level Config/ConfigObject patterns. However, a write-enabled production agent needs additional authorization, persistence and operational safeguards beyond query generation.

```
User request: "Change X to 0.0003"
        |
        v
Plan -> resolve exact object + attribute + type/range
        |
        v
Policy check / authorization / confirmation
        |
        v
C++ wrapper -> OksObject::SetAttributeValue(...)
        |
        v
Persist using the environment's supported save/update/commit path
        |
        v
Return audit record + repository version + resulting value
```

- Do not let the LLM issue shell commands or arbitrary C++ to perform a write.
- Require exact class/object/attribute resolution. Do not accept fuzzy matches at the mutation boundary.
- Use schema range/type/nullability checks before mutation.
- Log user intent, resolved target, previous value, new value, repository version and outcome.
- For running DAQ, distinguish changing the stored OKS configuration from applying/reloading that configuration in the live system. The supplied references document configuration publishing/reloading mechanisms but do not provide a universal deployment procedure for every installation.

14. Repository Versioning and Run Context

OKS configuration is git-backed in the documented ATLAS/TDAQ environment. The references describe version selectors by tag, hash and date, and identify repository APIs and environment variables used to resolve configuration versions.

Need	Recommended behavior
Current configuration	Use the current repository exposed by TDAQ_DB_REPOSITORY / deployment configuration.
Historical configuration	Resolve tag/hash/date to a specific repository version before loading data.
Run-specific question	Use run/partition mapping mechanisms when available; do not infer a version from natural language alone.
Answer provenance	Return repository version/hash/tag/date with results whenever possible.

```
Examples from the references:
oks_clone_repository --version tag:r380689@all_hosts
oks_clone_repository --version hash:6800fe3b
oks_clone_repository --version 'date:2020-08-02 10:00:00'
```

```
Programmatic version APIs include:
get_repository_versions()
get_repository_versions_by_hash(...)
get_repository_versions_by_date(...)
```

15. Training / Fine-Tuning Strategy

The supplied references explicitly report that the harvested corpus contains 50 gold-standard English -> OKS query rows, but also state that no official NL-to-OKS pairs were found. Therefore, the project should use the gold pairs as seed material and expand them with machine-checked examples against the live grammar and schema.

15.1 Do not train the model to memorize the schema inventory

- Schema names and objects can change with repository version.
- The references explicitly state that the concrete class inventory in the harvested material was incomplete.
- The live schema should remain the source of truth; RAG supplies current schema context.

15.2 What fine-tuning should teach

- Semantic decomposition of questions into class, attribute, relationship, ID, operator and value.
- Construction of the intermediate query tree.
- Correct use of this vs all.
- Correct use of some vs all in relationships.
- Correct nesting of and/or/not.
- Correct object-id placement inside relationships.
- Path-query reasoning.
- Refusal/clarification when the schema cannot uniquely resolve the request.

15.3 Training example format

```
{
  "question": "Which applications run on hostA?",
  "schema_context": "...relevant Application + RunsOn metadata...",
  "query_ir": { ... },
  "query_oks": "(all (\"RunsOn\" some (object-id \"hostA\" )))",
  "validation": {"parser": "pass", "schema": "pass"}
}
```

16. Retrieval and Indexing Strategy

16.1 Canonical ingestion pipeline

```
OKS repository
  -> parse *.schema.xml
  -> resolve includes and inheritance
  -> parse *.data.xml
  -> canonicalize metadata
  -> generate chunks
  -> lexical index + vector index + metadata filters
  -> retrieval service
```

16.2 Chunking policy

14. Class-level chunk: identity, description, abstract flag, superclasses.
15. One attribute chunk per attribute, linked to its class and source file.
16. One relationship chunk per relationship, including target class and cardinality.
17. One object summary chunk per object, but do not indiscriminately embed every attribute value if the dataset is large; prefer searchable exact fields plus selected semantic descriptions.
18. Query-example chunks stored separately and tagged by grammar construct.

16.3 Retrieval ranking

Priority	Signal
Highest	Exact class/attribute/relationship/object ID match.
High	Schema description semantic similarity.
High	Correct inheritance/source-file relationship.
Medium	Verified English->OKS example similarity.
Low	Generic query grammar chunks already present in the system prompt/context cache.

17. Error Model and Recovery

Error	Meaning	Agent action
oks::bad_query_syntax	Query/path could not be parsed.	Do not execute; repair IR/serialization and retry once.
oks::BadReqExp	Regex comparator failed to compile.	Reject or repair regex; never silently broaden.
oks::QueryFailed	Evaluation failure.	Return tool error with diagnostics; no invented answer.
oks_dump 2	Bad OKS file(s).	Repository/configuration problem.
oks_dump 3	Bad query.	Query generation/validation problem.
oks_dump 4	Class not found.	Schema retrieval or entity resolution problem.
oks_dump 5	Dangling references.	Repository integrity problem.
No unique target	Multiple candidate objects/attributes.	Ask for clarification or surface candidates; never guess at write boundary.

18. Testing and Evaluation

18.1 Unit tests

- scope: this/all
- all comparators
- = != ~= < <= > >=
- and/or/not operand counts
- object-id expressions
- some/all relationship nesting
- direct/nested path parsing
- quoted strings and escaping
- type/range validation
- class/attribute/relationship lookup

18.2 Integration tests

- NL -> retrieval -> IR -> OKS query -> parser -> execution -> JSON result
- schema change -> metadata re-index -> new query resolution
- repository version switch -> reproducible historical query
- invalid schema reference -> deterministic failure
- dangling reference -> safe failure
- write request -> authorization/policy block unless explicitly allowed

18.3 Metrics

Metric	Definition
Schema grounding accuracy	Correct class/attribute/relationship selection from live schema.
Query exact match	Generated OKS string exactly matches gold after canonicalization.
Query execution success	Generated query parses and executes successfully on authoritative OKS.
Semantic execution accuracy	Returned result set matches expected result set.
Abstention precision	Rate at which the agent correctly refuses when schema evidence is insufficient.
Write safety rate	Unauthorized or ambiguous writes blocked by policy layer.

19. Codex Agent Implementation Instructions

The following is the handoff specification for a coding agent. Treat the live OKS headers/repository as the final API authority if any signature differs from this document.

19.1 Phase 1 - Repository introspection

19. Detect installed OKS include/library paths and version.
20. Locate active repository via environment/configuration.
21. Implement a small metadata probe that loads schema/data and reports number of classes and files.
22. Do not modify the repository in this phase.

19.2 Phase 2 - Metadata service

23. Implement list_classes().
24. Implement describe_class(class).
25. Return inherited attributes/relationships as resolved by the OKS API.
26. Implement object enumeration/find_object for exact IDs.
27. Serialize canonical metadata JSON.
28. Persist a versioned metadata snapshot for reproducibility.

19.3 Phase 3 - RAG

29. Chunk canonical metadata, not raw XML only.
30. Store exact identifiers as lexical-searchable fields.
31. Embed descriptions/semantic text for vector retrieval.
32. Store source_file, repository_version, class_name and entity_type as metadata filters.
33. Add query-example documents from the supplied corpus, clearly marking illustrative examples.

19.4 Phase 4 - Query compiler

34. Define a JSON Schema for the intermediate query representation.
35. Implement IR validation independent of the LLM.

36. Implement serializer to canonical OKS S-expression.
37. Validate serialized query with the installed OksQuery parser / q.good() or direct construction.
38. Return both IR and serialized query for observability.

19.5 Phase 5 - Execution wrapper

39. Implement read-only query_objects and find_path first.
40. Normalize OksObject results into JSON without exposing raw pointers.
41. Include repository version/provenance in every response.
42. Add deterministic error codes.
43. Only after read path passes tests, implement write operations.

19.6 Phase 6 - MCP server

44. Expose tools with strict JSON schemas.
45. Keep tool names stable and semantically narrow.
46. Prevent the LLM from passing arbitrary shell/C++ code.
47. Pass retrieved schema context to the planning/query-generation step.
48. Require explicit policy checks for writes.

20. Recommended Repository Structure

```
oks-agent/
├── apps/
│   ├── oks_metadata_dump.cpp
│   ├── oks_query_runner.cpp
│   └── oks_write_runner.cpp
├── include/
│   └── oks_agent/
│       ├── metadata.hpp
│       ├── query_ir.hpp
│       ├── query_compiler.hpp
│       ├── executor.hpp
│       └── policy.hpp
├── python/
│   ├── mcp_server.py
│   ├── rag/
│   │   ├── ingest.py
│   │   ├── retrieve.py
│   │   └── context_builder.py
│   └── agent/
│       └── prompts.py
├── schema_index/
├── tests/
│   ├── query_grammar/
│   ├── schema_grounding/
│   ├── integration/
│   └── safety/
└── docs/
    └── generated-metadata/
```

21. Acceptance Criteria

- The system can load the actual repository schema and enumerate classes without hard-coded class names.
- The system can return a machine-readable description of a class, inherited attributes and relationships.
- The LLM receives retrieved schema context rather than relying on memorized schema names.
- The LLM produces an intermediate query tree/IR before final OKS serialization.
- The serialized query uses only documented operators and constructs.
- Every generated class/attribute/relationship is checked against the live schema before execution.
- Every read query is validated by the authoritative parser/API before execution.
- Execution results are returned as structured JSON with repository/version provenance.
- Invalid or ambiguous queries do not produce fabricated answers.
- Writes are isolated, auditable and policy-controlled.
- Historical questions resolve a specific repository/configuration version before query execution.
- The Codex implementation includes tests covering every core grammar construct and representative schema patterns.

22. Known Gaps and Source Limitations

- The supplied research states that the formal BNF is reconstructed because no formal grammar file was found; authoritative implementation sources are src/query.cpp, include/oks/query.hpp comments, QueryWindow documentation and release notes.
- The supplied research states that the concrete class/object inventory was incomplete because several large schema/data files were truncated during collection. Therefore, inventories in this document are examples, not exhaustive production inventories.
- The supplied research did not identify an official NL-to-OKS training corpus; the 50-row gold set is a curated research corpus, with some rows marked illustrative.
- The accessible references did not establish a complete end-to-end deployment recipe for every TDAQ environment. The installed OKS version and local operational tooling must be consulted.
- Run-to-revision mapping is documented conceptually, but the supplied material does not provide a universal production implementation path for all deployments.
- Write semantics and live DAQ reconfiguration are deployment-dependent; changing a stored OKS value is not automatically equivalent to applying that change to a running detector/DAQ system.

Appendix A. OKS Query Cheat Sheet

Construct	Canonical form	Meaning
Scope	(this <expr>)	Current class only
Scope	(all <expr>)	Class + subclasses
Attribute	("A" "V" =)	A equals V
Attribute	("A" "V" !=)	A differs from V
Attribute	("A" "V" ~=)	A matches regex V
Attribute	("A" "V" >)	A > V
Attribute	("A" "V" >=)	A >= V
Attribute	("A" "V" <)	A < V
Attribute	("A" "V" <=)	A <= V
Boolean	(and e1 e2 ...)	All operands true
Boolean	(or e1 e2 ...)	At least one true
Boolean	(not e)	Negate
Object ID	(object-id "ID" =)	Object identity
Relationship	("R" some <expr>)	At least one target matches

Relationship	("R" all <expr>)	Every target matches
Path	(path-to "id@class" (...))	Relationship path search

Appendix B. Source Traceability

The implementation specification was assembled from the user-supplied PDFs listed below. Specific source concepts are traceable to the following sections/pages in those PDFs:

Reference	Relevant sections/pages
OKS_DAL_Technical_Reference.pdf	pp. 4-16 system model, query grammar, C++/Python layer, DAL, versioning; pp. 17-20 worked schema/data examples; pp. 20-23 gold query corpus; pp. 34+ provenance/limitations.
OKS_Grammar_Query_CppAPI_Reference.pdf	pp. 4-7 grammar/parser/path; pp. 8+ XML DTD; pp. 13-15 C++ APIs; later pages for CLI, DAL, release history and examples.
OKS_Query_Language_Learning_Guide_Detailed(1).pdf	pp. 3-14 query mental model, semantic decomposition, path queries, programmatic API, validation, schema grounding and MCP architecture; pp. 15-16 limitations and cheat sheet.

Implementation authority

When the Codex agent encounters a mismatch between this specification and the installed OKS headers/repository, the installed code and live schema take precedence. The agent should report the mismatch, preserve the source version in logs, and avoid speculative compatibility behavior.